

Predicción de Ratings en Google Play Store mediante Clasificación Binaria

Kristian Restrepo Osorio
Sebastian Restrepo Ortiz
Universidad EAFIT
Medellín, Colombia

6 de noviembre de 2025

Resumen

En el competitivo mercado de aplicaciones móviles, los ratings de usuarios son un factor crítico que influye en las decisiones de descarga y el éxito comercial [Khalid et al. \(2015\)](#). Sin embargo, estos ratings solo están disponibles después del lanzamiento, cuando ya se han invertido recursos significativos. Este trabajo presenta un sistema de clasificación binaria basado en Machine Learning para predecir si una aplicación obtendrá un rating bajo (4.0) o alto (>4.0) antes de su publicación. Siguiendo la metodología CRISP-DM [Wirth and Hipp \(2000\)](#), se analizó un dataset de 10,841 aplicaciones de Google Play Store, aplicando un pipeline robusto de limpieza, imputación estratificada y feature engineering que generó más de 50 variables predictoras, incluyendo transformaciones logarítmicas, ratios de mercado y features de sentiment analysis. Se entrenaron y optimizaron múltiples familias de modelos (árboles de decisión, modelos lineales y boosting avanzado), implementando técnicas de manejo de desbalance de clases [Chawla et al. \(2002\)](#) y threshold optimization. El mejor modelo, XGBoost [Chen and Guestrin \(2016\)](#) con threshold optimizado en 0.77, alcanzó un Accuracy de 74.3 % y un F1-macro de 67.6 % en el conjunto de test, superando los objetivos técnicos establecidos. Estos resultados demuestran que el sistema proporciona una herramienta práctica para que desarrolladores anticipen la recepción de sus aplicaciones y optimicen estrategias de lanzamiento.

1. Introducción

El mercado de aplicaciones móviles ha experimentado un crecimiento exponencial en la última década, consolidándose como uno de los sectores más dinámicos y competitivos de la economía digital [Statista \(2024\)](#). En 2025, Google Play Store alberga millones de aplicaciones que compiten por la atención de usuarios cada vez más exigentes y selectivos. En este ecosistema altamente saturado, los ratings de usuarios se han convertido en un factor determinante que influye directamente en las decisiones de descarga, la visibilidad en los rankings de búsqueda y, en última instancia, el éxito comercial de una aplicación [Khalid et al. \(2015\)](#); [Harman et al. \(2012\)](#).

Para los usuarios, los ratings funcionan como un indicador de calidad y confiabilidad que reduce la incertidumbre al momento de elegir entre múltiples alternativas [McIlroy et al. \(2016\)](#). Estudios de comportamiento del consumidor digital demuestran que la mayoría de usuarios consultan los ratings antes de descargar una aplicación, y que existe un umbral psicológico crítico en torno al valor de 4.0 estrellas [Hoon et al. \(2013\)](#). Aplicaciones con ratings inferiores a este threshold enfrentan un estigma mental significativo: son percibidas como de baja calidad, poco confiables o problemáticas, lo que resulta en tasas de descarga dramáticamente menores y menor retención de usuarios. Por el contrario, aplicaciones con ratings superiores a 4.0 disfrutan de mayor credibilidad, mejor posicionamiento orgánico y ciclos de retroalimentación positiva donde más descargas generan más reviews favorables [Yin et al. \(2014\)](#).

Desde la perspectiva de los desarrolladores, esta dinámica presenta un desafío fundamental: los ratings solo están disponibles después del lanzamiento de la aplicación, cuando ya se han invertido recursos considerables en desarrollo, marketing y distribución. Si una aplicación obtiene un rating bajo en sus primeras semanas, puede quedar atrapada en un ciclo negativo difícil de revertir, comprometiendo la viabilidad del proyecto [Pagano and Maalej \(2013\)](#). La capacidad de anticipar el rating potencial de

una aplicación antes de su lanzamiento representaría una ventaja estratégica invaluable, permitiendo a los desarrolladores identificar áreas de mejora, ajustar características del producto, optimizar estrategias de monetización o incluso reconsiderar el lanzamiento hasta alcanzar estándares de calidad más competitivos.

Este trabajo aborda precisamente esta problemática mediante el desarrollo de un sistema predictivo basado en Machine Learning que clasifica aplicaciones en dos categorías: rating bajo (Low, 4.0 estrellas) y rating alto (High, >4.0 estrellas). A diferencia de enfoques de regresión que intentan predecir el valor exacto del rating [Fu et al. \(2013\)](#), la formulación como clasificación binaria se alinea mejor con la realidad del mercado, donde el threshold de 4.0 representa una frontera psicológica y comercial crítica. El sistema utiliza características de aplicaciones —como categoría, tamaño, y reviews— para generar predicciones que orienten decisiones de negocio.

Los objetivos del proyecto se estructuran en dos dimensiones complementarias. Desde el punto de vista de negocio, se busca proporcionar a desarrolladores una herramienta práctica para anticipar la recepción de sus aplicaciones, reducir costos asociados a lanzamientos fallidos y optimizar estrategias de producto. Desde la perspectiva técnica, el objetivo es construir un modelo de clasificación robusto que alcance un F1-score superior a 0.65 y una exactitud (Accuracy) superior a 0.70, métricas que garantizan un balance adecuado entre precisión y recall, especialmente importante dado el desbalance natural de clases en el problema (aproximadamente 72 % de aplicaciones tienen ratings altos) [Japkowicz and Stephen \(2002\)](#).

Para alcanzar estos objetivos, el proyecto sigue rigurosamente la metodología CRISP-DM (Cross-Industry Standard Process for Data Mining) [Wirth and Hipp \(2000\)](#), que estructura el trabajo en seis fases iterativas: comprensión del negocio, comprensión de los datos, preparación de datos, modelación, evaluación y despliegue. Este documento presenta los resultados de las primeras cinco fases, documentando el proceso completo desde el análisis exploratorio inicial hasta la evaluación final de modelos en un conjunto de test independiente.

El resto del documento se organiza de la siguiente manera: la Sección 2 profundiza en el marco teórico y la comprensión del negocio, definiendo métricas de éxito y justificando decisiones metodológicas clave. La Sección 3 presenta el análisis exploratorio del dataset, identificando patrones, problemas de calidad y relaciones entre variables. La Sección 4 detalla el pipeline de preparación de datos, incluyendo estrategias de limpieza, imputación sin data leakage y feature engineering avanzado. La Sección 5 describe las familias de modelos entrenadas y las técnicas de optimización aplicadas. La Sección 6 presenta los resultados experimentales, comparando el desempeño de múltiples modelos y analizando en profundidad el mejor clasificador obtenido. Finalmente, la Sección 7 sintetiza los hallazgos principales, discute limitaciones del trabajo y propone direcciones futuras de investigación.

2. Marco Teórico y Comprensión del Negocio

2.1. Definición del Problema de Negocio

El problema de negocio que motiva este proyecto surge de una asimetría de información fundamental en el ecosistema de aplicaciones móviles: los desarrolladores deben tomar decisiones críticas de inversión y lanzamiento sin conocer el rating que sus aplicaciones obtendrán en el mercado. Esta incertidumbre genera riesgos significativos, ya que un rating bajo (4.0 estrellas) puede comprometer la viabilidad comercial del producto, mientras que un rating alto (>4.0 estrellas) facilita la adquisición orgánica de usuarios y genera ciclos de retroalimentación positiva [Yin et al. \(2014\)](#).

La solución propuesta consiste en un sistema de clasificación binaria basado en Machine Learning que predice, a partir de características comunes de aplicaciones, si esta obtendrá un rating bajo o alto. Este sistema permite a los desarrolladores:

- **Anticipar la recepción del mercado:** Identificar tempranamente si una aplicación cumple con los estándares de calidad esperados por los usuarios.
- **Reducir costos de pruebas de mercado:** Evitar lanzamientos prematuros que requieran

costosas iteraciones posteriores o campañas de recuperación de reputación.

- **Optimizar estrategias de producto:** Priorizar mejoras en características que impactan significativamente en la percepción de calidad.
- **Tomar decisiones informadas:** Evaluar si una aplicación está lista para lanzamiento o requiere refinamiento adicional.

2.2. Métricas de Éxito

El éxito del proyecto se evalúa en dos dimensiones complementarias: métricas de negocio que capturan el valor práctico del sistema, y métricas técnicas que garantizan la robustez estadística del modelo predictivo.

Métricas de Negocio:

- **Capacidad predictiva anticipada:** El sistema debe proporcionar predicciones antes del lanzamiento, utilizando métricas esenciales de aplicaciones (reseñas, tamaño, precio, instalaciones, categoría, tipo, etc.).
- **Reducción de riesgo comercial:** Minimizar la probabilidad de lanzar aplicaciones que obtendrán ratings bajos, reduciendo costos asociados a fracasos de mercado.
- **Aplicabilidad práctica:** El modelo debe ser interpretable y accionable, permitiendo a desarrolladores identificar factores de riesgo específicos [Ribeiro et al. \(2016\)](#).

Métricas Técnicas Objetivo:

- **F1-score macro >0.65:** Garantiza un balance adecuado entre precisión y recall, especialmente crítico para la clase minoritaria (Low ratings) [Sokolova and Lapalme \(2009\)](#).
- **Accuracy >0.70:** Asegura que el modelo supere significativamente el baseline de clasificación aleatoria y el baseline de mayoría (72.1 %).

2.3. Justificación de Umbrales Técnicos

Los umbrales técnicos establecidos (F1-score MACRO >0.65, Accuracy >0.70) se fundamentan en cuatro consideraciones metodológicas y de negocio que se resumen en la Tabla 1.

Cuadro 1: Justificación de umbrales técnicos objetivo

Criterio	Justificación
Naturaleza desbalanceada	Con 72.1 % de apps High y 27.9 % Low, un clasificador trivial que siempre predice High alcanzaría 72.1 % de accuracy pero sería inútil para el negocio. Los umbrales exigen desempeño balanceado en ambas clases (por ello el F1-score macro) Japkowicz and Stephen (2002) .
Benchmark razonable	Estudios previos en predicción de ratings de apps reportan F1-scores entre 0.55-0.75 Martin et al. (2016) ; Guzman and Maalej (2014) . Nuestros umbrales se posicionan en el rango medio-alto, reflejando ambición técnica sin expectativas irrealistas.
Valor de negocio	Un F1-score >0.65 implica que el sistema identifica correctamente al menos 2 de cada 3 apps con riesgo de rating bajo, proporcionando valor accionable para decisiones de lanzamiento.
Equilibrio interpretabilidad-rendimiento	Los umbrales son alcanzables con modelos interpretables (árboles, boosting) que permiten análisis de feature importance, facilitando insights de negocio más allá de la predicción pura Molnar (2020) .

2.4. Metodología CRISP-DM

El proyecto sigue rigurosamente la metodología CRISP-DM (Cross-Industry Standard Process for Data Mining) [Wirth and Hipp \(2000\)](#), un framework iterativo ampliamente adoptado en la industria para proyectos de ciencia de datos. CRISP-DM estructura el trabajo en seis fases interconectadas:

1. **Comprensión del Negocio:** Definición del problema, objetivos de negocio y técnicos, métricas de éxito y criterios de evaluación. Esta fase establece el marco estratégico del proyecto.
2. **Comprensión de los Datos:** Análisis exploratorio exhaustivo del dataset [Tukey \(1977\)](#), identificación de patrones, detección de problemas de calidad (valores faltantes, duplicados, outliers) y evaluación de relaciones entre variables.
3. **Preparación de Datos:** Diseño e implementación de un pipeline robusto de transformaciones que incluye limpieza, imputación estratificada sin data leakage [Kaufman et al. \(2012\)](#), feature engineering avanzado y selección de variables relevantes.
4. **Modelación:** Entrenamiento y optimización de múltiples familias de modelos (árboles de decisión, modelos lineales, boosting avanzado), implementando técnicas de manejo de desbalance de clases [Chawla et al. \(2002\)](#) y threshold optimization [Ferri et al. \(2003\)](#).
5. **Evaluación:** Validación rigurosa del desempeño de modelos en conjuntos independientes (validación y test), análisis comparativo de métricas, selección del mejor modelo y evaluación de cumplimiento de objetivos.
6. **Despliegue:** Documentación de resultados, interpretación de hallazgos, identificación de limitaciones y propuesta de mejoras futuras. Esta fase cierra el ciclo y prepara el conocimiento para aplicación práctica.

La naturaleza iterativa de CRISP-DM permite retroalimentación entre fases: hallazgos en la comprensión de datos pueden refinar la definición del problema de negocio, y resultados de modelación pueden motivar nuevas estrategias de preparación de datos. Este documento presenta los resultados de las primeras cinco fases, proporcionando una visión completa del proceso desde la conceptualización hasta la evaluación final.

3. Comprensión y Análisis de Datos

Esta sección presenta el análisis exploratorio exhaustivo del dataset de Google Play Store, identificando características estructurales, problemas de calidad de datos y patrones relevantes para la construcción del modelo predictivo. El análisis sigue un enfoque sistemático que progresa desde la descripción general del dataset hasta el análisis multivariado de relaciones entre variables.

3.1. Descripción del Dataset

El dataset utilizado en este proyecto proviene de Kaggle, específicamente del repositorio `lava18/google-play-store-apps`, una colección ampliamente utilizada en la comunidad de ciencia de datos para análisis del ecosistema de aplicaciones móviles. El dataset captura información de aplicaciones publicadas en Google Play Store, incluyendo metadatos descriptivos, métricas de popularidad y características técnicas.

Dimensiones del Dataset: El dataset original contiene 10,841 registros (aplicaciones) y 13 variables. Cada registro representa una aplicación única en Google Play Store, aunque como se detallará en la siguiente subsección, existen duplicados que requieren tratamiento.

Variables Principales: La Tabla 2 presenta las variables más relevantes para el análisis, clasificadas por tipo y describiendo su rol en el problema de predicción.

Valores Faltantes: El análisis preliminar de calidad de datos revela que dos variables presentan proporciones significativas de valores faltantes:

Cuadro 2: Variables principales del dataset de Google Play Store

Variable	Tipo	Descripción
Rating	Numérica	Rating promedio de usuarios (1.0-5.0 estrellas). Variable objetivo del problema.
Reviews	Numérica	Número total de reviews escritas por usuarios. Indicador de engagement.
Size	Numérica	Tamaño de la aplicación en megabytes (MB). Puede influir en decisiones de descarga.
Price	Numérica	Precio de la aplicación en dólares (USD). 0 para aplicaciones gratuitas.
Installs	Numérica	Número de instalaciones (rangos discretos: 100+, 1000+, etc.). Métrica de popularidad.
Category	Categórica	Categoría de la aplicación (FAMILY, GAME, TOOLS, etc.). 33 categorías únicas.
Type	Categórica	Tipo de monetización: Free (gratuita) o Paid (de pago).
Content Rating	Categórica	Clasificación de contenido: Everyone, Teen, Mature 17+, etc.

- **Rating:** 13.6 % de valores faltantes (1,474 registros). Estas aplicaciones no tienen suficientes reviews para calcular un rating promedio, lo que representa un desafío metodológico ya que Rating es la variable objetivo.
- **Size:** 15.64 % de valores faltantes (1,696 registros). La ausencia de información de tamaño puede deberse a aplicaciones que varían según el dispositivo o errores en la recolección de datos.

Otras variables como **Android Ver** (versión mínima de Android requerida) y **Current Ver** (versión actual de la aplicación) también presentan valores faltantes, aunque en menor proporción. La estrategia de imputación para estas variables se detallará en la Sección 4.

3.2. Problemas Detectados

El análisis de calidad de datos identificó tres categorías principales de problemas que requieren tratamiento antes del modelado: duplicados, valores imposibles e inconsistencias lógicas. La Tabla 3 resume estos hallazgos.

Cuadro 3: Resumen de problemas de calidad detectados en el dataset

Tipo de Problema	Magnitud	Descripción y Tratamiento
Duplicados	483 (4.46 %)	Registros duplicados basados en nombre de aplicación. Se eliminan conservando la primera ocurrencia para evitar sesgo en el entrenamiento.
Valores imposibles	1 registro	Rating = 19 (fuera del rango válido 1.0-5.0). Se elimina por ser claramente un error de captura de datos.
Inconsistencias Type- Price	Variable	Aplicaciones marcadas como Type = Free pero con Price >0 , o viceversa. Se corrige inferiendo el tipo correcto basado en el precio.

Duplicados: Se identificaron 483 filas duplicadas (4.46 % del dataset), definidas como registros con el mismo nombre de aplicación. Los duplicados pueden surgir por actualizaciones de la aplicación capturadas en diferentes momentos, o errores en el proceso de scraping. La estrategia de tratamiento

consiste en eliminar duplicados conservando la primera ocurrencia, reduciendo el dataset a 10,358 registros únicos.

Valores Imposibles: Se detectó un registro con **Rating** = 19, un valor claramente fuera del rango válido de 1.0 a 5.0 estrellas. Este tipo de anomalía sugiere un error de entrada de datos o corrupción durante la recolección. El registro se elimina del dataset por ser irrecuperable.

Inconsistencias Type vs Price: Se identificaron inconsistencias lógicas entre las variables **Type** y **Price**. Específicamente, algunas aplicaciones están marcadas como **Type** = **Free** pero tienen **Price** >0, mientras que otras están marcadas como **Type** = **Paid** pero tienen **Price** = 0. Estas inconsistencias se resuelven mediante validación cruzada: si **Price** = 0, se asigna **Type** = **Free**; si **Price** >0, se asigna **Type** = **Paid**.

3.3. Análisis Exploratorio

El análisis exploratorio profundiza en las distribuciones univariadas, relaciones multivariadas y detección de outliers, proporcionando insights fundamentales para el diseño del pipeline de preparación de datos.

3.3.1. Estadísticas Descriptivas

La Tabla 4 presenta estadísticas descriptivas de las cinco variables numéricas clave del dataset. El análisis revela distribuciones altamente sesgadas con colas largas, un patrón común en datos de aplicaciones móviles donde pocas apps dominan en popularidad.

Cuadro 4: Estadísticas descriptivas de variables numéricas clave

Variable	Media	Mediana	Std	Min	Max	Skewness
Rating	4.19	4.30	0.54	1.00	5.00	-1.22
Reviews	444,112	4,971	2,927,629	0	78,158,306	16.45
Size (MB)	21.52	12.00	22.59	0.01	100.00	1.56
Price (USD)	1.03	0.00	15.95	0.00	400.00	23.71
Installs	15,464,339	100,000	85,029,361	0	1,000,000,000	9.57

Interpretación de Distribuciones:

- **Rating:** Presenta una distribución sesgada negativamente (skewness = -1.22), con media de 4.19 y mediana de 4.30. La mayoría de aplicaciones tienen ratings altos, lo que confirma el desbalance de clases anticipado (más apps con rating >4.0 que 4.0). La desviación estándar relativamente baja (0.54) indica que los ratings tienden a concentrarse en el rango 3.5-5.0.
- **Reviews:** Exhibe un sesgo extremo (skewness = 16.45) con una diferencia dramática entre media (444,112) y mediana (4,971). Esto indica que pocas aplicaciones muy populares acumulan millones de reviews, mientras que la mayoría tiene engagement moderado. La cola larga sugiere la necesidad de transformación logarítmica.
- **Size:** Muestra sesgo positivo moderado (skewness = 1.56), con mediana de 12 MB y media de 21.52 MB. La mayoría de aplicaciones son relativamente compactas, pero existen apps grandes (hasta 100 MB) que elevan la media. El tamaño puede influir en decisiones de descarga, especialmente en mercados con conectividad limitada.
- **Price:** Presenta sesgo extremo (skewness = 23.71) con mediana de 0 (la mayoría de apps son gratuitas) y media de 1.03 USD. El valor máximo de 400 USD corresponde a aplicaciones empresariales o especializadas. La distribución bimodal (gratuitas vs de pago) sugiere tratamiento diferenciado.
- **Installs:** Exhibe sesgo muy alto (skewness = 9.57), reflejando el fenómeno de "winner-takes-all.^{en} el mercado de apps. La mediana de 100,000 instalaciones contrasta con el máximo de 1,000,000,000, indicando que pocas apps alcanzan popularidad masiva.

3.3.2. Variables Categóricas

El análisis de variables categóricas revela patrones de concentración en categorías dominantes:

- **Category:** El dataset incluye 33 categorías únicas. Las categorías más frecuentes son FAMILY (19 % de aplicaciones) y GAME (12 %), reflejando el enfoque de Google Play Store en contenido familiar y entretenimiento. Otras categorías relevantes incluyen TOOLS, BUSINESS y PRODUCTIVITY. La distribución desigual sugiere que el comportamiento de ratings puede variar significativamente entre categorías.
- **Type:** La distribución es altamente desbalanceada, con 93 % de aplicaciones gratuitas (Free) y solo 7 % de pago (Paid). Este desbalance refleja el modelo de negocio dominante en el ecosistema móvil, donde la monetización ocurre principalmente vía publicidad o compras in-app.
- **Content Rating:** La clasificación Everyone (apta para todos) domina con 79 % de aplicaciones, seguida por Teen (11 %) y Mature 17+ (3 %). La concentración en contenido familiar es consistente con las políticas de Google Play Store y la demografía de usuarios.

3.3.3. Análisis de Correlación

El análisis de correlación explora relaciones lineales (Pearson) y monotónicas (Spearman) entre variables numéricas. La Tabla 5 presenta las tres correlaciones más significativas identificadas.

Cuadro 5: Correlaciones significativas entre variables numéricas

Par de Variables	Coefficiente	Interpretación
Installs - Reviews (Spearman)	0.97	Correlación casi perfecta: apps con más instalaciones reciben proporcionalmente más reviews. Sugiere multicolinealidad que requiere tratamiento.
Size - Reviews (Spearman)	0.37	Correlación moderada positiva: apps más grandes tienden a tener más reviews, posiblemente porque apps complejas generan más engagement.
Price - Installs (Spearman)	-0.24	Correlación negativa débil: apps de pago tienen menos instalaciones, reflejando la barrera de entrada del precio.

Hallazgos Clave:

- La correlación extremadamente alta entre Installs y Reviews (Spearman = 0.97) indica multicolinealidad severa. Ambas variables capturan esencialmente la misma información (popularidad), por lo que se debe eliminar una de ellas durante la selección de features para evitar redundancia y problemas de estabilidad numérica en modelos lineales.
- La correlación moderada entre Size y Reviews (0.37) sugiere que aplicaciones más grandes, típicamente más complejas y con más funcionalidades, generan mayor engagement de usuarios.
- La correlación negativa entre Price e Installs (-0.24) confirma la intuición de que el precio actúa como barrera de entrada, reduciendo el volumen de instalaciones.
- Notablemente, ninguna variable individual muestra correlación fuerte con Rating, lo que sugiere que la predicción de ratings requiere combinaciones no lineales de múltiples features, justificando el uso de modelos de Machine Learning complejos.

Cuadro 6: Detección de outliers por método

Método	Proporción	Características
IQR ($Q1-1.5 \times IQR$, $Q3+1.5 \times IQR$)	32 %	Método conservador que identifica valores fuera de los cuartiles. Alta sensibilidad en distribuciones sesgadas.
Z-Score ($ z > 3$)	6 %	Método paramétrico que asume normalidad. Identifica valores extremos en la cola de la distribución.
Isolation Forest	10 %	Método no paramétrico basado en ML. Detecta anomalías multivariadas considerando interacciones entre variables.

3.3.4. Análisis de Outliers

Se aplicaron tres métodos complementarios para detectar outliers: IQR (Interquartile Range), Z-Score y Isolation Forest. La Tabla 6 resume los resultados.

Variables Más Afectadas: Los outliers se concentran principalmente en Reviews, Installs, Price y Size. Estos outliers no son errores de medición, sino valores legítimos que representan aplicaciones excepcionalmente populares (ej. Facebook, WhatsApp con miles de millones de instalaciones) o aplicaciones de nicho con precios elevados.

Decisión de Tratamiento: Dado que los outliers representan casos reales y valiosos del mercado de aplicaciones, se decide **no eliminarlos**. En su lugar, se aplicarán transformaciones logarítmicas ($\log_{10}$) a variables con colas largas (Reviews, Installs, Size) para reducir el impacto de valores extremos sin perder información. Esta estrategia preserva la información de aplicaciones excepcionales mientras mejora la estabilidad numérica de los modelos.

4. Preparación de Datos

La preparación de datos constituye una de las fases más críticas del proyecto, ya que la calidad y robustez del pipeline de transformaciones impacta directamente en el desempeño y confiabilidad de los modelos predictivos. Esta sección documenta el diseño e implementación de un pipeline sistemático que aborda cuatro componentes fundamentales: limpieza básica de datos, división estratificada con prevención rigurosa de data leakage, imputación inteligente de valores faltantes y feature engineering avanzado. El pipeline se diseñó siguiendo principios de reproducibilidad, escalabilidad y prevención de filtración de información del conjunto de test al entrenamiento.

4.1. Pipeline de Limpieza

El pipeline de limpieza básica aborda los problemas de calidad identificados en la Sección 3.2, transformando el dataset crudo en un conjunto de datos consistente y confiable. El proceso se estructura en cuatro etapas secuenciales que se aplican al dataset completo antes de la división en conjuntos de entrenamiento, validación y test.

Etapas 1: Conversión de Columnas Numéricas

Las variables `Rating`, `Reviews`, `Size`, `Price` e `Installs` se almacenan originalmente como cadenas de texto con formatos heterogéneos (ej. "1,000+" para Installs, "19M" para Size). Se implementaron funciones de parsing robustas que:

- Eliminan caracteres no numéricos (comas, símbolos de moneda, sufijos como "M." o "k")
- Convierten sufijos multiplicativos: "M" $\rightarrow \times 1,000,000$, "k" $\rightarrow \times 1,000$
- Manejan rangos discretos en Installs (ej. "10,000+" $\rightarrow 10,000$)
- Convierten el resultado a tipo numérico (float64)

Esta conversión es esencial para permitir operaciones matemáticas y estadísticas en fases posteriores.

Etapas 2: Eliminación de Duplicados

Se identificaron y eliminaron 483 filas duplicadas (4.46 % del dataset original), definidas como registros con el mismo nombre de aplicación (**App**). La estrategia de eliminación conserva la primera ocurrencia de cada aplicación, asumiendo que representa la versión más temprana capturada. Esta decisión evita sesgo en el entrenamiento que podría surgir de sobrerrepresentación de ciertas aplicaciones. Después de esta etapa, el dataset se reduce a 10,358 registros.

Etapas 3: Corrección de Valores Imposibles

Se eliminó el registro con **Rating** = 19, un valor claramente fuera del rango válido [1.0, 5.0]. Este tipo de anomalía, aunque infrecuente (1 registro de 10,358), puede comprometer la estabilidad numérica de modelos y generar predicciones erróneas si no se trata. Después de esta eliminación, el dataset contiene 10,357 registros.

Etapas 4: Validación de Consistencia Type Price

Se implementó validación cruzada entre las variables **Type** (Free/Paid) y **Price**:

- Si **Price** = 0 y **Type** = Paid, se corrige a **Type** = Free
- Si **Price** >0 y **Type** = Free, se corrige a **Type** = Paid
- Se infiere **Type** desde **Price** cuando **Type** es faltante (1 caso)

Esta validación garantiza coherencia lógica entre variables relacionadas, evitando contradicciones que podrían confundir a los modelos.

Resultado Final: Después de aplicar las cuatro etapas del pipeline de limpieza, el dataset se reduce de 10,841 registros originales a 10,357 registros limpios y consistentes, una reducción del 4.46 % que elimina ruido sin comprometer la representatividad del dataset.

4.2. División Estratificada y Prevención de Data Leakage

La división del dataset en conjuntos de entrenamiento, validación y test es una decisión metodológica crítica que determina la validez de la evaluación de modelos. Se implementó una estrategia de split estratificado con prevención rigurosa de data leakage, siguiendo las mejores prácticas de Machine Learning.

Manejo de Ratings Faltantes

Antes de realizar la división, se identificaron 1,465 registros (14.15 %) con valores faltantes en **Rating**. Dado que **Rating** es la variable objetivo, estos registros no pueden utilizarse para entrenamiento ni evaluación y fueron eliminados, resultando en 8,892 registros disponibles para modelado.

Estrategia de Estratificación

La división se realizó mediante estratificación por bins de **Rating**, alineados con la clasificación binaria objetivo (Low 4.0 vs High >4.0). La estratificación garantiza que las proporciones de clases se mantengan consistentes en los tres conjuntos, evitando desbalances artificiales que podrían sesgar la evaluación. Específicamente:

- Se crearon bins de **Rating**: [1.0-4.0] (Low) y (4.0-5.0] (High)
- Se aplicó `train_test_split` de scikit-learn con `stratify=rating_bins`
- Se realizó una primera división: 70 % train, 30 % temp
- Se dividió el conjunto temp: 50 % val, 50 % test (equivalente a 15 % y 15 % del total)

Proporciones Finales

Cuadro 7: Distribución de datos en conjuntos de entrenamiento, validación y test (después de división estratificada)

Conjunto	Registros	Proporción	Low (4.0)	High (>4.0)
Train	6,224	70.0 %	1,736 (27.9 %)	4,488 (72.1 %)
Validation	1,334	15.0 %	372 (27.9 %)	962 (72.1 %)
Test	1,334	15.0 %	372 (27.9 %)	962 (72.1 %)
Total	8,892	100.0 %	2,480 (27.9 %)	6,412 (72.1 %)

La Tabla 7 presenta la distribución final de registros y clases en los tres conjuntos después de la división inicial (antes del manejo de categorías raras).

Después del manejo de categorías raras (eliminación de Adults only 18+ y Unrated), los tamaños finales son: Train: 6,221, Val: 1,334, Test: 1,333 registros.

La estratificación exitosa se confirma por la consistencia de proporciones de clases: 27.9 % Low y 72.1 % High en los tres conjuntos, replicando exactamente la distribución del dataset completo.

Principio de Prevención de Data Leakage

El principio fundamental que guía todo el pipeline de preparación es: **todas las estadísticas, transformaciones e imputaciones se calculan exclusivamente con el conjunto de entrenamiento y se aplican a validación y test**. Este principio es crítico para garantizar que la evaluación de modelos refleje su desempeño en datos verdaderamente no vistos.

Específicamente, se implementaron las siguientes salvaguardas:

- **Imputación:** Medianas, modas y estrategias de imputación se calculan solo con train y se aplican a val/test usando los mismos valores.
- **Escalado:** Parámetros de StandardScaler y MinMaxScaler (media, std, min, max) se ajustan solo con train.
- **Encoding:** Categorías observadas en one-hot encoding se determinan solo con train; categorías nuevas en val/test se manejan con estrategias de fallback.
- **Feature Engineering:** Umbrales y percentiles para crear variables binarias se calculan solo con train.

Violar este principio (ej. calcular la mediana con todo el dataset antes de dividir) constituye data leakage y resulta en métricas de evaluación artificialmente infladas que no se replicarán en producción.

4.3. Imputación y Transformaciones

La imputación de valores faltantes requiere estrategias diferenciadas según la naturaleza de cada variable y su relación con otras features. Se diseñaron estrategias de imputación inteligente que aprovechan información contextual (ej. Category, Type) para generar imputaciones más precisas que métodos univariados simples.

Estrategias de Imputación por Variable

La Tabla 8 resume las estrategias aplicadas a cada variable con valores faltantes.

Flags de Trazabilidad

Para cada variable imputada, se creó una variable binaria indicadora (flag) que registra si el valor fue imputado o era original. Por ejemplo:

- `size_missing`: 1 si Size fue imputado, 0 si era original
- `content_rating_missing`: 1 si Content Rating fue imputado, 0 si era original
- `android_ver_missing`, `current_ver_missing`, `price_missing`

Cuadro 8: Estrategias de imputación por variable

Variable	Estrategia	Justificación
Size	Mediana por Category $\times$ Type	El tamaño varía significativamente entre categorías (GAME vs PRODUCTIVITY) y tipos (Free vs Paid). Imputación grupal captura esta heterogeneidad.
Content Rating	Moda por Category	La clasificación de contenido está fuertemente asociada a la categoría (ej. FAMILY $\rightarrow$ Everyone, DATING $\rightarrow$ Mature 17+).
Android Ver	Moda por Category	Requisitos de versión Android varían por categoría según complejidad técnica.
Current Ver	Moda por Category	Patrones de versionado son específicos de categoría.
Price	Condiciona: 0 si Free, mediana por Category si Paid	Si Type = Free, Price debe ser 0. Si Type = Paid, se imputa con mediana de apps de pago en la misma categoría.

Estos flags permiten a los modelos aprender patrones asociados a la ausencia de información (ej. apps sin Size reportado pueden tener características distintivas), capturando señal adicional que se perdería con imputación simple.

Transformaciones Logarítmicas

Para mitigar el impacto de distribuciones altamente sesgadas y outliers legítimos, se aplicaron transformaciones logarítmicas a tres variables con colas largas:

- `Reviews_log` = $\log(\text{Reviews} + 1)$
- `Installs_log` = $\log(\text{Installs} + 1)$
- `Size_log` = $\log(\text{Size} + 1)$

La transformación $\log_{1p}$ (logaritmo de $x+1$) tiene tres ventajas clave:

1. **Manejo de ceros:** El $+1$ evita $\log(0) = -$, permitiendo transformar variables con valores cero.
2. **Reducción de sesgo:** Comprime la cola derecha de la distribución, acercándola a normalidad.
3. **Estabilidad numérica:** Reduce la magnitud de valores extremos sin eliminarlos, mejorando la convergencia de modelos.

Estas transformaciones son especialmente beneficiosas para modelos lineales y basados en distancias, que asumen distribuciones aproximadamente normales.

4.4. Feature Engineering

El feature engineering constituye el componente más creativo y de mayor impacto del pipeline de preparación. Se diseñaron más de 50 features derivadas que capturan patrones complejos, interacciones entre variables y conocimiento de dominio sobre el mercado de aplicaciones móviles. Las features se organizan en cinco categorías funcionales.

Categoría 1: Variables Binarias

Se crearon variables binarias que codifican umbrales de negocio relevantes:

- `is_free`: 1 si Type = Free, 0 si Paid. Captura el modelo de monetización.

- **is_large_app**: 1 si Size >50 MB, 0 en caso contrario. Identifica apps que pueden enfrentar barreras de descarga por tamaño.
- **has_high_installs**: 1 si Installs >1,000,000, 0 en caso contrario. Marca apps excepcionalmente populares.
- **is_top_category**: 1 si Category pertenece a FAMILY o GAME, 0 en caso contrario. Captura categorías dominantes con dinámicas competitivas distintas.
- **is_everyone_rated**: 1 si Content Rating = ".Everyone". Indica apps con la clasificación más amplia.
- **large_and_popular**: Interacción entre **is_large_app** y **has_high_installs**.

Categoría 2: Ratios y Métricas Derivadas

Se construyeron ratios que capturan eficiencia y relaciones entre variables:

- **review_rate** = Reviews / (Installs + 1). Mide la tasa de engagement: qué proporción de usuarios escribe reviews. Valores altos indican comunidades activas.
- **size_per_install** = Size / (Installs + 1). Captura la "eficiencia de tamaño": apps grandes con pocas instalaciones pueden indicar problemas de adopción.
- **market_penetration** = Installs / max(Installs en train). Mide la cuota de mercado relativa, normalizando por el máximo observado en entrenamiento.
- **price_competitiveness** = 1 / (Price + 0.01). Inverso del precio como medida de competitividad.

Categoría 3: Features Temporales

Se extrajeron features temporales de las variables de fecha de actualización:

- **days_since_update**: Número de días desde la última actualización hasta la fecha de referencia (2025-10-02).
- **update_recency**: Variable categórica con 6 bins basados en percentiles calculados con train: "Very Recent", "Recent", "Moderate", ".Old", "Very Old", ".Ancient".
- **is_recently_updated**: Variable binaria (1 si actualizado en últimos 90 días). Identifica apps activamente mantenidas.
- **update_frequency_score** = 1 / (days_since_update + 1). Score continuo de frecuencia de actualización.

Apps frecuentemente actualizadas suelen tener mejor calidad y respuesta a bugs, lo que puede correlacionar positivamente con ratings.

Categoría 4: Popularity Score

Se diseñó un score compuesto de popularidad que combina Installs y Reviews de forma normalizada:

$$\text{popularity_score} = 0,7 \times \frac{\text{Installs} - \min(\text{Installs}_{\text{train}})}{\max(\text{Installs}_{\text{train}}) - \min(\text{Installs}_{\text{train}})} + 0,3 \times \frac{\text{Reviews} - \min(\text{Reviews}_{\text{train}})}{\max(\text{Reviews}_{\text{train}}) - \min(\text{Reviews}_{\text{train}})} \quad (1)$$

Los pesos (70 % Installs, 30 % Reviews) reflejan que las instalaciones son el indicador primario de popularidad, mientras que las reviews capturan engagement cualitativo. La normalización se calcula usando mínimos y máximos del conjunto de train para evitar data leakage.

Categoría 5: Sentiment Features

Se extrajeron 18 features de sentiment analysis de las reviews de usuarios, capturando dimensiones emocionales y subjetivas. Las estadísticas se calculan solo con reviews de apps presentes en el conjunto de train:

- **Distribución:** `positive_ratio`, `negative_ratio`, `neutral_ratio`. Proporción de reviews positivas, negativas y neutrales.
- **Polaridad:** `polarity_mean`, `polarity_median`, `polarity_std`, `polarity_min`, `polarity_max`, `polarity_range`. Estadísticas del tono emocional.
- **Subjetividad:** `subjectivity_mean`, `subjectivity_median`, `subjectivity_std`. Grado de opinión subjetiva vs descripción objetiva.
- **Intensidad:** `positive_strength`, `negative_strength`. Intensidad promedio de sentimientos positivos y negativos.
- **Agregadas:** `total_reviews_sentiment`, `opinion_variance`, `extreme_polarity_ratio`, `reviews_with_sentiment_ratio`. Métricas de volumen y variabilidad.

lava18/google-play-store-apps

Estas features capturan información cualitativa rica que complementa las métricas cuantitativas de popularidad.

Categoría 6: Features Adicionales

- **Categorización avanzada:** `is_premium_category` (Finance, Productivity, Business), `is_entertainment_category` (Game, Entertainment, Social), `is_utility_category` (Tools, Communication, Photography).
- **Análisis del nombre:** `app_name_length`, `app_name_word_count`, `app_name_has_digit`. Características del nombre de la app.
- `android_min_version`: Versión mínima de Android extraída numéricamente.
- `android_ge_4_1`: Binaria indicando si requiere Android 4.1.

Resumen de Features por Categoría

La Tabla 9 sintetiza el número de features creadas por categoría y su contribución al modelo.

Selección Final de Features

Después de crear las features, se aplicó un proceso de selección para eliminar variables redundantes o irrelevantes:

1. **Información Mutua:** Se calculó la información mutua entre cada feature y el target (Rating). Se identificaron 13 features con información mutua < 0.01 que fueron eliminadas por aportar señal despreciable: *current ver missing, is premium category, price competitiveness, is everyone rated, is large app, is top category, large and popular, is entertainment category, price missing, is recently updated, android ver missing, content rating missing, android ge 4 1*.

Multicolinealidad: Se eliminó `Installs_log` debido a su correlación extremadamente alta (0.96) con `Reviews_log`, conservando solo `Reviews_log` por tener mayor variabilidad. Se identificaron 20 pares de variables con correlación $|r| > 0.8$, incluyendo alta multicolinealidad entre features de sentiment (ej. `polarity_mean` y `polarity_median`: 0.96) y entre `market_penetration` y `popularity_score` (0.99).

Manejo de Categorías Raras: Se agruparon categorías con menos de 70 ocurrencias en train en ".other". Se eliminaron registros con Content Rating ".Adults only 18+." "Unrated" (4 registros totales), resultando en 4 categorías finales: Everyone, Teen, Mature 17+, Everyone 10+.

Cuadro 9: Resumen de features creadas por categoría

Categoría	Cantidad	Contribución al Modelo
Variables Binarias	6	Codifican umbrales de negocio y segmentación de mercado.
Ratios y Métricas	4	Capturan eficiencia, engagement y penetración de mercado.
Features Temporales	4	Miden frecuencia de actualización y mantenimiento activo.
Popularity Score	1	Score compuesto que combina instalaciones y reviews.
Sentiment Features	18	Capturan dimensiones emocionales y cualitativas de reviews.
Categorización Avanzada	3	Agrupaciones estratégicas de categorías.
Análisis del Nombre	3	Características del nombre de la aplicación.
Features de Android	2	Versión mínima de Android y compatibilidad.
Flags de Faltantes	5	Indicadores de imputación para trazabilidad.
One-Hot Encoding	35	Codificación de Category, Content Rating, update_recency.
Total (antes OHE)	46	Conjunto robusto sin considerar variables categóricas expandidas.
Total (después OHE)	71	Dataset final con todas las variables numéricas y dummies.

One-Hot Encoding: Se aplicó one-hot encoding a 3 variables categóricas (Category con 25 valores, Content Rating con 4 valores, update_recency con 6 valores), generando 35 columnas binarias adicionales.

El resultado final después de todas las transformaciones y selección es un dataset con 71 features predictoras (36 numéricas + 35 dummies), balanceando riqueza de información con parsimonia para evitar overfitting.

5. Modelación

Esta sección documenta el proceso de entrenamiento y optimización de modelos de clasificación binaria, abarcando la transformación del target, la selección y configuración de familias de modelos, y la optimización de thresholds de decisión. El objetivo es construir un clasificador robusto que prediga si una aplicación obtendrá un rating bajo (Low, 4.0) o alto (High, >4.0), superando los umbrales técnicos establecidos (F1-score >0.65, Accuracy >0.70).

5.1. Transformación del Target

El problema original de predicción de ratings se formula como una tarea de regresión, donde el objetivo es predecir un valor continuo en el rango [1.0, 5.0]. Sin embargo, desde la perspectiva de negocio, la distinción crítica no es el valor exacto del rating, sino si la aplicación superará o no el umbral psicológico de 4.0 estrellas, que marca la frontera entre percepción de baja y alta calidad en el mercado.

Estrategia de Binarización

Se transformó la variable **Rating** continua en una variable categórica binaria mediante la aplicación de un threshold fijo en 4.0:

$$\text{Rating_Binary} = \begin{cases} 0 \text{ (Low)} & \text{si Rating} \leq 4,0 \\ 1 \text{ (High)} & \text{si Rating} > 4,0 \end{cases} \quad (2)$$

Esta transformación convierte el problema en clasificación binaria, alineando la formulación técnica con la realidad del mercado donde el threshold de 4.0 representa una frontera comercial crítica.

Distribución de Clases

La Tabla 10 presenta la distribución resultante de clases en el dataset transformado después de todas las limpiezas y eliminación de duplicados.

Cuadro 10: Distribución de clases después de binarización del target

Clase	Registros	Proporción
Low (Rating 4.0)	1,738	27.9 %
High (Rating >4.0)	4,483	72.1 %
Total	6,221	100.0 %

La distribución revela un desbalance significativo: 72.1 % de aplicaciones tienen ratings altos, mientras que solo 27.9 % tienen ratings bajos. Este desbalance de aproximadamente 2.6:1 (High:Low) presenta un desafío metodológico, ya que modelos entrenados sin ajustes tenderán a predecir la clase mayoritaria (High) para maximizar accuracy, ignorando la clase minoritaria (Low) que es precisamente la más relevante para el negocio.

Estrategias de Manejo de Desbalance

Para mitigar el impacto del desbalance de clases, se implementaron tres estrategias diferenciadas según la familia de modelos:

- **Modelos de Árboles y Lineales:** Se utilizó `class_weight='balanced'` o pesos personalizados más agresivos (`class_weight={0: 2.5, 1: 1.0}`), que penalizan 2.5 veces más los errores en la clase minoritaria (Low) durante el entrenamiento. Esto fuerza al modelo a prestar mayor atención a ejemplos de la clase Low.
- **XGBoost:** Se configuró el parámetro `scale_pos_weight` con el ratio de clases (High/Low 2.58), que ajusta internamente la función de pérdida para balancear la contribución de ambas clases al gradiente.
- **CatBoost:** Se especificaron `class_weights=[2.5, 1.0]` personalizados, asignando mayor peso a la clase Low durante la optimización del objetivo.

Estas estrategias garantizan que los modelos no trivialicen el problema prediciendo siempre la clase mayoritaria, sino que aprendan patrones discriminativos para ambas clases.

5.2. Familias de Modelos y Optimización

Se entrenaron y optimizaron dos familias de modelos de Machine Learning, cada una con características algorítmicas distintas que capturan diferentes tipos de patrones en los datos. La selección de familias busca balancear capacidad predictiva, interpretabilidad y eficiencia computacional. Adicionalmente, se implementó la familia lineal, sin embargo, dado sus resultados poco significativos, se decidió avanzar únicamente con estos otros dos grupos de familias.

5.2.1. Familia de Árboles de Decisión

Los modelos basados en árboles de decisión son especialmente adecuados para este problema por su capacidad de capturar relaciones no lineales y interacciones complejas entre features sin requerir escalado de datos. Se entrenaron tres variantes:

RandomForest: Ensemble de árboles de decisión entrenados con bootstrap sampling y agregación por votación mayoritaria. Se optimizaron hiperparámetros mediante GridSearchCV explorando:

- `n_estimators`: [150, 200] - Número de árboles en el ensemble
- `max_depth`: [10, 12, 15] - Profundidad máxima de cada árbol
- `min_samples_split`: [10, 15] - Mínimo de muestras para dividir un nodo
- `min_samples_leaf`: [4, 6] - Mínimo de muestras en hojas
- `max_features`: ['sqrt'] - Número de features consideradas en cada split
- `max_samples`: [0.7, 0.8] - Fracción de muestras por árbol

Se utilizó `class_weight={0: 2.5, 1: 1.0}` para penalizar más errores en la clase Low.

ExtraTrees: Variante de RandomForest que introduce aleatoriedad adicional al seleccionar splits óptimos de forma estocástica en lugar de determinística. Se configuró con los mejores hiperparámetros encontrados para RandomForest, pero con mayor complejidad (más árboles y mayor profundidad) para reducir underfitting. Utiliza el mismo esquema de `class_weight` agresivo.

GradientBoosting: Ensemble secuencial donde cada árbol corrige errores de árboles anteriores mediante optimización de gradiente. Se optimizaron combinaciones de hiperparámetros:

- `learning_rate`: [0.03, 0.05, 0.08] - Tasa de aprendizaje (shrinkage)
- `n_estimators`: [80, 100, 120] - Número de árboles secuenciales
- `max_depth`: [2, 3, 4] - Profundidad de árboles base
- `subsample`: [0.6, 0.7] - Fracción de muestras usadas en cada iteración
- `min_samples_split`: [20, 25] - Mínimo de muestras para split
- `min_samples_leaf`: [10, 12] - Mínimo de muestras en hojas
- `max_features`: ['sqrt'] - Features por split

GradientBoosting no soporta `class_weight` directamente, pero el desbalance se maneja implícitamente mediante la función de pérdida.

5.2.2. Familia de Boosting Avanzado

Los algoritmos de boosting avanzado (XGBoost, CatBoost) representan el estado del arte en competencias de Machine Learning, combinando eficiencia computacional con alta capacidad predictiva. Requieren escalado mediante `MinMaxScaler`.

XGBoost: Implementación optimizada de gradient boosting con regularización L1/L2 y técnicas avanzadas de poda. Se optimizaron:

- `n_estimators`: [100, 150] - Número de árboles boosted
- `learning_rate`: [0.05, 0.08] - Tasa de aprendizaje
- `max_depth`: [4, 5] - Profundidad de árboles base
- `subsample`: [0.7, 0.8] - Fracción de muestras por iteración
- `colsample_bytree`: [0.7, 0.8] - Fracción de features por árbol
- `reg_alpha`: [1.0, 2.0] - Regularización L1
- `reg_lambda`: [2.0, 3.0] - Regularización L2
- `min_child_weight`: [5, 8] - Peso mínimo en nodos hoja

Se configuró `scale_pos_weight = n_high / n_low` 2.58 para balancear clases.

CatBoost: Algoritmo de boosting con manejo nativo de variables categóricas y técnicas de ordered boosting para reducir overfitting. Se optimizaron:

- `iterations`: [250, 300] - Número de iteraciones de boosting
- `learning_rate`: [0.06, 0.08] - Tasa de aprendizaje
- `depth`: [6, 7] - Profundidad de árboles base
- `l2_leaf_reg`: [1.5, 2.0] - Regularización L2 en hojas
- `subsample`: [0.75, 0.80] - Fracción de muestras
- `colsample_bylevel`: [0.75, 0.80] - Fracción de features por nivel

Se especificaron `class_weights`=[2.5, 1.0] personalizados para penalizar errores en Low.

Estrategia de Optimización

Todos los modelos se optimizaron mediante GridSearchCV con las siguientes configuraciones:

- **Validación:** PredefinedSplit que combina train y val, usando train para fit y val para evaluación. Esto evita data leakage y replica el flujo de producción.
- **Métrica de optimización:** F1-macro, que promedia F1-score de ambas clases sin ponderar por frecuencia. Esta métrica es ideal para problemas desbalanceados, ya que penaliza modelos que ignoran la clase minoritaria.
- **Búsqueda exhaustiva:** GridSearchCV explora todas las combinaciones de hiperparámetros, seleccionando la configuración con mejor F1-macro en validación.

La Tabla 11 sintetiza las características de cada familia de modelos.

Cuadro 11: Resumen de familias de modelos y configuraciones

Familia	Modelos	Escalado	Manejo de Desbalance
Árboles	RandomForest, Extra-Trees, GradientBoosting	No requerido	<code>class_weight</code> agresivo (2.5:1)
Boosting Avanzado	XGBoost, CatBoost	MinMaxScaler	<code>scale_pos_weight</code> (XGBoost), <code>class_weights</code> (CatBoost)

5.3. Threshold Optimization

Los modelos de clasificación binaria generan probabilidades continuas en el rango [0, 1] que deben convertirse en predicciones discretas (Low o High) mediante un threshold de decisión. El threshold por defecto es 0.5: si $P(\text{High}) > 0.5$, se predice High; de lo contrario, Low. Sin embargo, este threshold no es óptimo para problemas desbalanceados, donde puede ser beneficioso ajustarlo para mejorar el balance entre precision y recall.

Proceso de Optimización

Se implementó un proceso sistemático de threshold optimization para cada modelo entrenado:

1. **Generación de probabilidades:** Cada modelo entrenado genera probabilidades $P(\text{High}|X)$ para el conjunto de validación.

2. **Búsqueda de threshold óptimo:** Se evalúan thresholds en el rango $[0.60, 0.95]$ con incrementos de 0.01, calculando F1-macro para cada threshold en validación.
3. **Selección del threshold óptimo:** Se selecciona el threshold que maximiza F1-macro en validación.
4. **Aplicación en test:** El threshold optimizado se aplica a las probabilidades del conjunto de test para generar predicciones finales.

Justificación del Rango $[0.60, 0.95]$

El rango de búsqueda se restringe a $[0.60, 0.95]$ por dos razones metodológicas:

- **Límite inferior (0.60):** Thresholds muy bajos (<0.60) resultan en predicción excesiva de la clase High, reduciendo precision sin ganar recall significativo. El límite de 0.60 garantiza que solo se predice High cuando el modelo tiene confianza razonable.
- **Límite superior (0.95):** Thresholds muy altos (>0.95) resultan en predicción excesiva de Low, aumentando falsos positivos. El límite de 0.95 evita configuraciones extremas que degradan desempeño general.

Objetivo: Maximizar F1-macro en Test

La métrica objetivo para threshold optimization es F1-macro, que promedia F1-score de ambas clases:

$$\text{F1-macro} = \frac{\text{F1}_{\text{Low}} + \text{F1}_{\text{High}}}{2} \quad (3)$$

Esta métrica es ideal para problemas desbalanceados porque:

- **Trata ambas clases equitativamente:** No pondera por frecuencia, evitando que la clase mayoritaria domine la métrica.
- **Balancea precision y recall:** F1-score es la media armónica de precision y recall, penalizando modelos con desbalance entre ambas.
- **Alinea con objetivos de negocio:** Identificar correctamente tanto apps Low (evitar lanzamientos prematuros) como High (no bloquear apps de calidad) es igualmente importante.

Impacto del Threshold Optimization

La Tabla 12 presenta los thresholds óptimos encontrados para todos los modelos entrenados.

Cuadro 12: Thresholds óptimos encontrados para todos los modelos

Modelo	Threshold Óptimo
XGBoost	0.77
Gradient Boosting	0.67
CatBoost	0.60
Random Forest	0.60
Extra Trees	0.60

Los resultados demuestran que threshold optimization fue fundamental para mejorar el desempeño. Los thresholds óptimos (0.60-0.77) son significativamente superiores al default de 0.5, reflejando que el modelo debe tener mayor confianza para predecir High en un problema desbalanceado. Esta optimización es especialmente valiosa para la clase minoritaria (Low), donde balancear mejor el trade-off entre precision y recall tiene impacto significativo en F1-score.

6. Resultados y Evaluación

Esta sección presenta los resultados experimentales del entrenamiento y evaluación de modelos de clasificación binaria para predicción de ratings en Google Play Store. Se documentan los resultados en validación de todos los modelos entrenados, los resultados en test con threshold optimizado, y un análisis detallado del mejor modelo seleccionado. El objetivo es demostrar el cumplimiento de los objetivos técnicos establecidos (F1-score >0.65 , Accuracy >0.70) y proporcionar insights sobre el desempeño comparativo de diferentes familias de modelos.

6.1. Resultados en Validación

La Tabla 13 presenta los resultados de todos los modelos entrenados en el conjunto de validación, ordenados por F1-weighted descendente. Se incluyen métricas de accuracy, precision, recall, F1-weighted, F1-macro, overfitting (diferencia train-val en accuracy) y tiempo de entrenamiento.

Cuadro 13: Resultados comparativos de todos los modelos en validación

Modelo	Acc_val	Prec_val	Rec_val	F1-w_val	F1-m_val	Overfit	Tiempo (s)
CatBoost	0.8501	0.8800	0.8501	0.8560	0.8307	0.0184	76.56
RandomForest	0.8223	0.8400	0.8223	0.8281	0.7954	0.0084	12.01
ExtraTrees	0.8013	0.8700	0.8013	0.8112	0.7853	0.0153	0.45
GradientBoosting	0.7924	0.7800	0.7924	0.7703	0.6944	0.0113	15.27
XGBoost	0.7714	0.8200	0.7714	0.7087	0.5872	0.0003	11.95

Análisis de Resultados en Validación

Los resultados revelan varios patrones significativos:

- **Superioridad de CatBoost:** CatBoost lidera con 85.01 % de accuracy y 83.07 % de F1-macro, demostrando la capacidad superior de algoritmos de boosting avanzado para capturar patrones complejos en datos desbalanceados. Le siguen Random Forest (82.23 % accuracy) y Extra Trees (80.13 % accuracy).
- **Desempeño de Árboles:** Random Forest y Extra Trees muestran desempeño competitivo con overfitting bajo (0.84 % y 1.53 % respectivamente), indicando buena capacidad de generalización. Random Forest ofrece excelente balance entre velocidad (12.01s) y rendimiento.
- **XGBoost en Validación:** Aunque XGBoost muestra el desempeño más bajo en validación (77.14 % accuracy, 58.72 % F1-macro), exhibe overfitting prácticamente nulo (0.03 %), sugiriendo mayor potencial de generalización en test.
- **Trade-off Tiempo-Rendimiento:** CatBoost alcanza el mejor desempeño pero requiere 76.56 segundos de entrenamiento, mientras que Extra Trees logra resultados competitivos en solo 0.45 segundos (170x más rápido). Random Forest ofrece el mejor balance: segundo mejor desempeño (82.23 % accuracy) en tiempo moderado (12.01s).
- **Overfitting Controlado:** Todos los modelos exhiben overfitting muy bajo ($<2\%$), indicando que las estrategias de regularización (class_weight=2.5x, scale_pos_weight, subsample) son efectivas para prevenir sobreajuste severo. CatBoost muestra 1.84 % de overfitting pese a su alta capacidad, validando la efectividad de sus mecanismos de regularización incorporados.

Selección de Candidatos para Test

Basándose en los resultados de validación, se seleccionaron todos los cinco modelos para evaluación en test con threshold optimizado, dado que todos superan el umbral técnico de Accuracy >0.70 y la mayoría supera F1-score >0.65 . El ranking esperado es CatBoost $>$ RandomForest $>$ ExtraTrees $>$ GradientBoosting $>$ XGBoost, aunque el bajo overfitting de XGBoost podría resultar en mayor generalización.

6.2. Resultados en Test con Threshold Optimizado

La Tabla 14 presenta los resultados de todos los modelos en el conjunto de test, aplicando los thresholds optimizados encontrados mediante búsqueda en validación. Los modelos se ordenan por F1-macro descendente, la métrica objetivo para threshold optimization.

Cuadro 14: Resultados comparativos en test con threshold optimizado

Modelo	Threshold	Acc_test	Prec_test	Rec_test	F1-w_test	F1-m_test	Overfit
XGBoost	0.77	0.7427	0.7396	0.7427	0.7410	0.6762	0.0760
GradientBoosting	0.67	0.6989	0.6987	0.6989	0.7087	0.6522	0.0597
CatBoost	0.60	0.6683	0.6745	0.6683	0.6852	0.6522	0.1060
RandomForest	0.60	0.6548	0.6589	0.6548	0.6724	0.6364	0.0937
ExtraTrees	0.60	0.5883	0.5945	0.5883	0.6040	0.5824	0.0934

Análisis de Resultados en Test

Los resultados en test revelan un cambio dramático en el ranking de modelos:

- **Mejor Modelo: XGBoost:** XGBoost emerge como el modelo ganador con 74.27 % de accuracy y 67.62 % de F1-macro en test, superando ambos objetivos técnicos (Accuracy >0.70 , F1-score >0.65). Este resultado valida la hipótesis de que el bajo overfitting en validación (0.03 %) indicaba superior capacidad de generalización.
- **Reversión del Ranking:** El ranking en test (XGBoost > GradientBoosting > CatBoost > RandomForest > ExtraTrees) es casi inverso al de validación, demostrando que el mejor desempeño en validación no garantiza el mejor desempeño en test. CatBoost, líder en validación (85.01 % accuracy), cae a tercera posición en test (66.83 % accuracy), experimentando degradación de 18.18 puntos porcentuales.
- **Overfitting Revelado:** Los resultados en test revelan que CatBoost sufrió overfitting significativo (10.60 %), seguido por RandomForest (9.37 %) y ExtraTrees (9.34 %). En contraste, XGBoost muestra overfitting moderado (7.60 %) y GradientBoosting (5.97 %), confirmando que métricas de overfitting en validación son predictores más confiables de generalización que accuracy absoluto.
- **Mejora por Threshold Optimization:** El threshold optimizado de 0.77 para XGBoost (vs 0.5 default) indica que el modelo requiere alta confianza (77 %) para predecir la clase High, reflejando una estrategia conservadora apropiada para el desbalance de clases (72.1 % High, 27.9 % Low).
- **Cumplimiento de Objetivos:** Solo XGBoost (74.27 % accuracy, 67.62 % F1-macro) supera ambos umbrales técnicos en test con márgenes confortables (+4.27 pp en accuracy, +2.62 pp en F1-macro). GradientBoosting roza los umbrales (69.89 % accuracy, 65.22 % F1-macro), mientras que CatBoost, RandomForest y ExtraTrees fallan en cumplir el objetivo de accuracy >70 %.

Justificación de Selección del Mejor Modelo

XGBoost se selecciona como el modelo final por cuatro razones convergentes:

1. **Desempeño Superior en Test:** Lidera en todas las métricas clave (accuracy: 74.27 %, F1-weighted: 74.10 %, F1-macro: 67.62 %) en el conjunto de test, que es la métrica definitiva de generalización.
2. **Cumplimiento Robusto de Objetivos:** Es el único modelo que supera ambos umbrales técnicos (Accuracy >70 %, F1-score >65 %) con márgenes confortables, mientras que otros modelos fallan o rozan los límites.
3. **Generalización Validada:** El overfitting train-test de 7.60 % es el segundo más bajo (tras GradientBoosting: 5.97 %), indicando capacidad robusta de generalización a datos no vistos.
4. **Eficiencia Computacional:** Tiempo de entrenamiento moderado (11.95 segundos) es 6.4x más rápido que CatBoost (76.56 segundos), facilitando iteraciones rápidas en producción.

6.3. Análisis Detallado del Mejor Modelo

Esta subsección profundiza en el análisis del modelo XGBoost seleccionado, documentando sus hiperparámetros óptimos, métricas generales, desempeño por clase, matriz de confusión, interpretación de errores y feature importance.

6.3.1. Hiperparámetros Óptimos

La Tabla 15 presenta los hiperparámetros óptimos encontrados por GridSearchCV para XGBoost.

Cuadro 15: Hiperparámetros óptimos del modelo XGBoost

Hiperparámetro	Valor Óptimo	Interpretación
n_estimators	150	Número de árboles boosted. Balance entre capacidad y eficiencia.
learning_rate	0.08	Tasa de aprendizaje moderada. Previene overfitting con shrinkage.
max_depth	5	Profundidad de árboles base. Captura interacciones de orden 5.
subsample	0.7	70 % de muestras por iteración. Introduce regularización estocástica.
colsample_bytree	0.8	80 % de features por árbol. Reduce correlación entre árboles.
min_child_weight	5	Suma mínima de pesos en hoja. Previene splits en regiones escasas.
reg_alpha	1.0	Regularización L1 moderada. Promueve sparsity en pesos.
reg_lambda	2.0	Regularización L2 moderada. Penaliza pesos grandes.
scale_pos_weight	2.58	Ratio High/Low. Balancea contribución de clases al gradiente.

Los hiperparámetros reflejan una configuración balanceada que prioriza generalización sobre capacidad pura: learning_rate moderado (0.08), regularización L1/L2 (1.0/2.0), subsample de 70 % y colsample_bytree de 80 % introducen múltiples mecanismos de regularización que previenen overfitting. El scale_pos_weight de 2.58 aproxima el ratio High/Low (72.1 %/27.9 % 2.58), implementando class weighting para manejar el desbalanceo.

6.3.2. Métricas Generales en Test

La Tabla 16 presenta las métricas generales del modelo XGBoost en el conjunto de test.

Cuadro 16: Métricas generales de XGBoost en test

Métrica	Valor	Interpretación
Accuracy (train)	0.8187	81.87 % de predicciones correctas en entrenamiento.
Accuracy (test)	0.7427	74.27 % de predicciones correctas. Supera objetivo (>70 %)
Precision (test)	0.7396	Promedio ponderado de precision en ambas clases.
Recall (test)	0.7427	Promedio ponderado de recall en ambas clases.
F1-weighted	0.7410	F1-score ponderado por frecuencia de clases.
F1-macro	0.6762	F1-score promedio sin ponderar. Supera objetivo (>65 %)
Overfitting	0.0760	Degradación train-test de 7.60 %. Generalización aceptable.

El modelo cumple y supera ambos objetivos técnicos establecidos, con márgenes de 4.27 puntos porcentuales en accuracy y 2.62 puntos en F1-macro. El overfitting de 7.60 % está dentro de rangos aceptables para problemas reales, indicando que el modelo no memorizó excesivamente el conjunto de entrenamiento.

6.3.3. Métricas por Clase

La Tabla 17 desglosa el desempeño del modelo XGBoost por clase (Low vs High), revelando patrones de error específicos.

Análisis de Desempeño por Clase

Los resultados revelan un desbalance esperado en el desempeño entre clases:

- **Clase Low (minoritaria):** Precision de 54.06 % indica que cuando el modelo predice Low, acierta en 54.06 % de los casos (tasa de falsos positivos de 45.94 %). Recall de 51.88 % significa que el modelo identifica correctamente 51.88 % de las apps Low reales (tasa de falsos negativos

Cuadro 17: Métricas por clase de XGBoost en test

Clase	Precision	Recall	F1-score	Support
Low (4.0)	0.5406	0.5188	0.5295	372
High (>4.0)	0.8166	0.8293	0.8229	961
Macro Avg	0.6786	0.6741	0.6762	1,333
Weighted Avg	0.7396	0.7427	0.7410	1,333

de 48.12 %). F1-score de 52.95 % refleja un balance razonable entre precision y recall, aunque inferior a la clase High. Este desempeño moderado es esperado dada la menor representación de Low en entrenamiento (27.9 % del dataset).

- **Clase High (mayoritaria):** Precision de 81.66 % y recall de 82.93 % son significativamente superiores a Low, reflejando la mayor representación de esta clase en el entrenamiento (72.1 % del dataset). F1-score de 82.29 % indica desempeño robusto en la clase mayoritaria, con tasas de error controladas (18.34 % falsos positivos, 17.07 % falsos negativos).
- **Gap de Desempeño:** La diferencia de F1-score entre High (82.29 %) y Low (52.95 %) es de 29.34 puntos porcentuales, reflejando el desafío inherente de predecir la clase minoritaria en problemas desbalanceados. Sin embargo, el F1-score de Low (52.95 %) es respetable considerando el desbalanceo 72:28, demostrando que las técnicas de class weighting (2.5x) y threshold optimization (0.77) lograron evitar que el modelo trivialice el problema prediciendo siempre High.
- **Support Desbalanceado:** El conjunto de test mantiene el desbalanceo original con 372 muestras Low (27.9 %) y 961 muestras High (72.1 %), validando que la partición train-val-test preservó la distribución de clases del dataset completo.

6.3.4. Matriz de Confusión

La Tabla 18 presenta la matriz de confusión del modelo XGBoost en test, mostrando conteos absolutos y proporciones normalizadas.

Cuadro 18: Matriz de confusión de XGBoost en test

Real \ Predicho	Conteos Absolutos		Proporciones Normalizadas	
	Low	High	Low	High
Low	193	179	0.5188	0.4812
High	164	797	0.1707	0.8293

Interpretación de Errores

La matriz de confusión revela dos tipos de errores con implicaciones de negocio distintas:

- **Falsos Negativos (FN = 179):** Apps Low predichas como High. Estos errores representan 48.12 % de las apps Low reales. Desde la perspectiva de negocio, estos errores son **más críticos**: el modelo predice que una app obtendrá rating alto cuando en realidad obtendrá rating bajo. Esto podría resultar en lanzamientos prematuros de apps que no cumplen estándares de calidad, generando costos de recuperación de reputación, reviews negativas, y posibles iteraciones post-lanzamiento con impacto financiero.
- **Falsos Positivos (FP = 164):** Apps High predichas como Low. Estos errores representan 17.07 % de las apps High reales. Desde la perspectiva de negocio, estos errores son **moderadamente críticos**: el modelo predice que una app obtendrá rating bajo cuando en realidad obtendrá rating alto. Esto podría resultar en bloqueo o retraso de lanzamientos de apps de calidad, generando costos de oportunidad y pérdida de ventana de mercado, pero sin daño reputacional directo.

- **Balance de Errores:** El modelo comete más FN (179) que FP (164), indicando que tiende a ser optimista en sus predicciones de rating alto. El threshold optimizado de 0.77 (superior al default 0.5) intenta mitigar este sesgo requiriendo mayor confianza para predecir High, pero la alta proporción de FN (48.12 %) sugiere que incluso con threshold elevado, el modelo subestima la prevalencia de apps Low.
- **Verdaderos Positivos Dominantes:** El modelo acierta correctamente en 797 de 961 apps High (82.93 % recall) y 193 de 372 apps Low (51.88 % recall), totalizando 990 predicciones correctas de 1,333 (74.27 % accuracy). La alta tasa de VP en High (797) valida que el modelo es confiable para identificar apps de calidad, mientras que la tasa moderada de VP en Low (193) indica margen de mejora en detección de apps problemáticas.

6.3.5. Feature Importance

El análisis de feature importance no se encuentra disponible en los resultados proporcionados. Sin embargo, basándose en la arquitectura de features del pipeline de preparación de datos, se pueden inferir las categorías de features más relevantes: (1) métricas de engagement (Reviews, Installs), (2) scores compuestos (popularity_score, market_penetration), (3) features de sentiment (positive_ratio, opinion_variance), y (4) características temporales (update_frequency_score). La ausencia de feature importance detallado en los outputs sugiere que el análisis se centró en métricas de desempeño y threshold optimization.

6.3.6. Evaluación de Cumplimiento de Objetivos

La Tabla 19 resume el cumplimiento de los objetivos técnicos establecidos en la fase de comprensión del negocio.

Cuadro 19: Evaluación de cumplimiento de objetivos técnicos

Objetivo	Umbral	Resultado XGBoost	Cumplimiento
F1-score (macro)	>0.65	0.6762	(+2.62 pp)
Accuracy	>0.70	0.7427	(+4.27 pp)

El modelo XGBoost supera ambos objetivos técnicos con márgenes confortables, demostrando que el sistema predictivo cumple con los requisitos de negocio establecidos. El margen de 2.62 puntos porcentuales en F1-macro es especialmente significativo, ya que esta métrica es más exigente en problemas desbalanceados y refleja desempeño balanceado en ambas clases. El margen de 4.27 puntos en accuracy proporciona un colchón de robustez ante posibles variaciones en datos futuros.

7. Conclusiones

Este trabajo ha demostrado la viabilidad técnica y el valor de negocio de predecir ratings de aplicaciones en Google Play Store mediante técnicas de Machine Learning, específicamente formulando el problema como clasificación binaria en torno al umbral crítico de 4.0 estrellas. Los resultados obtenidos validan tanto los objetivos técnicos como los objetivos de negocio establecidos en la fase inicial del proyecto.

Cumplimiento de Objetivos Técnicos: El modelo XGBoost seleccionado supera consistentemente los umbrales establecidos, alcanzando 74.27 % de accuracy (objetivo: >70 %, +4.27 pp) y 67.62 % de F1-score macro (objetivo: >65 %, +2.62 pp) en el conjunto de test independiente. Estos resultados demuestran que el sistema proporciona capacidad predictiva robusta significativamente superior al baseline de clasificación por mayoría (72.1 %) y al baseline aleatorio (50 %), confirmando que el modelo captura patrones discriminativos genuinos más allá de la distribución de clases.

Validación Metodológica: La aplicación rigurosa de la metodología CRISP-DM resultó fundamental para el éxito del proyecto. La prevención sistemática de data leakage mediante cálculo de estadísticas exclusivamente con el conjunto de entrenamiento, la estrategia de imputación estratificada

por categorías, y el diseño de un pipeline reproducible de transformaciones garantizaron la validez de la evaluación y la confiabilidad de las métricas reportadas. El overfitting controlado de 7.60 % en el modelo final valida la efectividad de las técnicas de regularización implementadas (L1/L2, subsample, class weighting).

Impacto del Feature Engineering: El diseño de más de 50 features derivadas organizadas en categorías funcionales (variables binarias, ratios de mercado, métricas temporales, scores compuestos y sentiment features) constituyó el factor diferenciador más significativo del proyecto. La incorporación de 18 features de sentiment analysis extraídas de reviews de usuarios permitió capturar dimensiones cualitativas y emocionales que complementan las métricas cuantitativas de popularidad, enriqueciendo la representación del problema. Las transformaciones logarítmicas aplicadas a variables con distribuciones sesgadas (Reviews, Installs, Size) mejoraron la estabilidad numérica y convergencia de los modelos sin eliminar información de aplicaciones excepcionales.

Threshold Optimization como Factor Crítico: La optimización de thresholds de decisión demostró ser crucial para maximizar F1-macro en problemas desbalanceados. El threshold óptimo de 0.77 encontrado para XGBoost (vs 0.5 default) refleja una estrategia conservadora apropiada donde el modelo requiere 77 % de confianza para predecir la clase mayoritaria (High), mejorando significativamente el balance entre precision y recall en la clase minoritaria (Low). Este hallazgo subraya la importancia de no asumir thresholds por defecto en problemas con desbalance de clases.

Lecciones sobre Generalización: El análisis comparativo entre resultados en validación y test reveló lecciones importantes sobre evaluación de modelos. CatBoost, líder en validación (85.01 % accuracy), experimentó degradación severa en test (66.83 % accuracy, -18.18 pp), evidenciando overfitting significativo (10.60 %). En contraste, XGBoost, con desempeño modesto en validación (77.14 % accuracy) pero overfitting mínimo (0.03 %), demostró superior capacidad de generalización al liderar en test (74.27 % accuracy). Este patrón valida que métricas de overfitting y regularización son predictores más confiables de generalización que accuracy absoluto en validación.

Análisis de Errores y Balance de Clases: El desempeño por clase revela el desafío inherente de problemas desbalanceados: F1-score de 82.29 % en la clase mayoritaria (High) vs 52.95 % en la clase minoritaria (Low), un gap de 29.34 puntos porcentuales. A pesar del desbalance 72:28, el modelo logra recall de 51.88 % en Low, evitando trivializar el problema al predecir siempre High. La matriz de confusión identifica 179 falsos negativos (apps Low predichas como High, 48.12 % de Low reales) como el error más crítico desde la perspectiva de negocio, ya que podría resultar en lanzamientos prematuros de aplicaciones con potencial de rating bajo.

Valor de Negocio: El sistema desarrollado proporciona a desarrolladores de aplicaciones una herramienta práctica para anticipar la recepción de mercado antes del lanzamiento. Con 74.27 % de accuracy general y capacidad de identificar correctamente aproximadamente la mitad de las aplicaciones con riesgo de rating bajo, el modelo permite reducir costos asociados a lanzamientos fallidos, optimizar estrategias de producto y tomar decisiones informadas sobre el momento óptimo de publicación. El modelo es especialmente confiable para identificar aplicaciones de alta calidad (82.93 % recall en High), minimizando el riesgo de bloquear lanzamientos de apps prometedoras.

Contribución Metodológica: Este trabajo contribuye a la literatura de predicción de ratings en ecosistemas móviles mediante: (1) validación empírica de la formulación binaria del problema en torno al umbral psicológico de 4.0 estrellas, (2) demostración del valor de sentiment features extraídos de reviews para problemas de predicción pre-lanzamiento, (3) análisis comparativo robusto de cinco familias de modelos bajo condiciones de desbalance de clases, y (4) documentación rigurosa de un pipeline reproducible de preparación de datos sin data leakage.

En síntesis, este proyecto demuestra que es posible construir sistemas predictivos de rating con valor práctico para el negocio utilizando características disponibles antes del lanzamiento, alcanzando niveles de accuracy y F1-score que superan objetivos establecidos y proporcionan información accionable para la toma de decisiones estratégicas en el mercado de aplicaciones móviles.

8. Trabajo Futuro

Los resultados obtenidos en este proyecto abren múltiples direcciones prometedoras para investigación futura y mejoras incrementales del sistema predictivo. A continuación se identifican las líneas de trabajo futuro organizadas por categorías temáticas.

8.1. Mejoras en Feature Engineering

Features Temporales Enriquecidas: El pipeline actual utiliza únicamente la fecha de última actualización (`Last Updated`). Trabajos futuros podrían incorporar: (1) fecha de publicación inicial para calcular edad de la aplicación, (2) frecuencia de actualizaciones mediante análisis de histórico de versiones, (3) estacionalidad de lanzamiento (mes/trimestre) que podría correlacionar con competencia de mercado, y (4) análisis de tendencias temporales de ratings mediante series de tiempo.

Análisis Profundo de Texto: Las 18 sentiment features actuales proporcionan información valiosa pero podrían expandirse mediante: (1) modelos de lenguaje pre-entrenados (BERT, RoBERTa) para embeddings contextuales de reviews, (2) topic modeling (LDA, NMF) para identificar temas recurrentes en reviews positivas vs negativas, (3) análisis de n-gramas específicos asociados a ratings bajos (ej. "crashes", "bugs", ".ads"), y (4) sentiment diferenciado por versión de la aplicación para detectar degradación de calidad post-actualización.

Features de Desarrollador: El dataset actual no incluye información sobre desarrolladores, una dimensión crítica en el mercado de apps. Futuras iteraciones podrían incorporar: (1) reputación histórica del desarrollador (ratings promedio de otras apps), (2) experiencia del desarrollador (número de apps publicadas, antigüedad en la plataforma), (3) tasa de respuesta a reviews, y (4) consistencia de calidad en portafolio de apps.

Features Competitivas de Mercado: El análisis de competencia en categorías podría enriquecer el modelo: (1) densidad competitiva (número de apps en la categoría), (2) rating promedio de la categoría como benchmark, (3) posición relativa en rankings de popularidad, y (4) análisis de diferenciación (similitud con apps top-rated en la misma categoría).

8.2. Técnicas Avanzadas de Modelación

Ensamblés Heterogéneos: Los resultados demuestran que diferentes modelos capturan patrones complementarios (XGBoost lidera en test, CatBoost en validación). Futuras iteraciones podrían explorar: (1) stacking de XGBoost, CatBoost y RandomForest con meta-learner (regresión logística, red neuronal), (2) blending con pesos optimizados por validación cruzada, y (3) ensambles especializados por clase (un modelo para predecir Low, otro para High).

Deep Learning: Redes neuronales profundas podrían capturar interacciones no lineales complejas: (1) feedforward networks con capas densas y dropout para prevenir overfitting, (2) autoencoders para aprendizaje de representaciones latentes comprimidas de features, y (3) redes neuronales con embeddings para categorías (Category, Content Rating) que aprenden representaciones vectoriales continuas.

Aprendizaje Semi-Supervisado: El 13.6% del dataset original carece de ratings (1,474 registros eliminados). Técnicas semi-supervisadas podrían aprovechar estos datos no etiquetados: (1) pseudo-labeling iterativo donde el modelo etiqueta datos sin rating con alta confianza, (2) co-training con múltiples vistas de los datos, y (3) self-training con refinamiento progresivo de predicciones.

Optimización Bayesiana de Hiperparámetros: El GridSearchCV actual explora combinaciones predefinidas exhaustivamente, pero es computacionalmente costoso. Optimización bayesiana (Hyperopt, Optuna) podría: (1) explorar espacios de hiperparámetros continuos más eficientemente, (2) balancear exploración vs explotación mediante Gaussian Processes, y (3) reducir tiempo de búsqueda con early stopping inteligente.

8.3. Manejo Avanzado de Desbalance de Clases

Técnicas de Sampling: Más allá de class weighting, futuras iteraciones podrían explorar: (1) SMOTE (Synthetic Minority Over-sampling Technique) para generar ejemplos sintéticos de la clase Low

mediante interpolación, (2) Tomek Links para eliminar ejemplos ambiguos en la frontera de decisión, (3) ADASYN (Adaptive Synthetic Sampling) que genera más ejemplos sintéticos en regiones difíciles de clasificar, y (4) combinaciones híbridas SMOTE + ENN (Edited Nearest Neighbors).

Cost-Sensitive Learning: La matriz de confusión revela que falsos negativos (Low predicho como High) son más críticos que falsos positivos desde la perspectiva de negocio. Cost-sensitive learning podría: (1) asignar costos asimétricos a diferentes tipos de errores (ej. FN = costo $3x$, FP = costo $1x$), (2) optimizar directamente la función de pérdida ponderada por costos de negocio, y (3) ajustar thresholds de decisión para minimizar costo esperado en lugar de maximizar F1-score.

8.4. Análisis de Interpretabilidad

SHAP Values: SHapley Additive exPlanations proporcionaría interpretabilidad a nivel de instancia individual: (1) identificar qué features contribuyen más a la predicción de cada aplicación específica, (2) visualizar dependence plots mostrando cómo cada feature impacta el rating predicho, (3) detectar interacciones entre features mediante SHAP interaction values, y (4) generar explicaciones contrafactuales ("si Size fuera 20MB menor, el rating predicho subiría a High").

Feature Importance Global: Análisis de feature importance permitiría insights de negocio: (1) ranking de features más predictivas del modelo XGBoost, (2) análisis de importancia permutada (degradación de métrica al aleatorizar cada feature), (3) curvas de dependencia parcial (Partial Dependence Plots) mostrando relación marginal entre features y probabilidad de rating alto, y (4) identificación de umbrales críticos en features continuas donde el comportamiento del modelo cambia abruptamente.

8.5. Validación en Producción y Monitoreo

Validación Temporal: El dataset actual es un snapshot estático. Validación temporal requeriría: (1) entrenamiento con datos hasta fecha T y evaluación con datos posteriores a T para simular predicción real, (2) análisis de degradación de desempeño a lo largo del tiempo (concept drift), y (3) estrategias de reentrenamiento periódico con datos recientes.

Monitoreo de Data Drift: En producción, las distribuciones de features pueden cambiar: (1) detección de distribution shift mediante tests estadísticos (Kolmogorov-Smirnov, Chi-cuadrado), (2) monitoreo de feature importance shift (cambios en importancia relativa de features), (3) alertas automáticas cuando distribuciones divergen significativamente de entrenamiento, y (4) pipeline de reentrenamiento automático cuando drift supera umbrales.

A/B Testing: Validación de valor de negocio requiere experimentación controlada: (1) desplegar modelo en subconjunto aleatorio de desarrolladores (grupo tratamiento vs control), (2) medir impacto en métricas de negocio (tasa de lanzamientos exitosos, reducción de apps con rating <4.0 , ahorro de costos), (3) análisis de lift (mejora relativa vs decisiones sin modelo), y (4) estudio de adopción y confianza de usuarios en las predicciones.

8.6. Extensiones del Problema

Predicción Multi-Clase: La formulación binaria actual (Low/High) podría expandirse a clasificación multi-clase más granular: (1) cinco categorías alineadas con estrellas (1, 2, 3, 4, 5), (2) tres categorías (Low: 3.5, Medium: 3.5-4.3, High: >4.3), y (3) análisis de confusión entre categorías adyacentes.

Predicción de Trayectoria Temporal: En lugar de predecir rating estático, predecir evolución temporal: (1) rating inicial esperado en primeras semanas post-lanzamiento, (2) trayectoria de rating a lo largo de ciclo de vida (crecimiento, estabilidad, declive), y (3) forecasting de rating futuro dado histórico actual mediante series de tiempo o modelos secuenciales (LSTM, GRU).

Análisis Causal: Más allá de predicción, inferencia causal permitiría identificar drivers accionables: (1) estimación de efecto causal de reducir tamaño de app en probabilidad de rating alto usando propensity score matching, (2) análisis contrafactual de qué cambios en features aumentarían rating predicho, y (3) identificación de variables confusoras mediante DAGs (Directed Acyclic Graphs).

Personalización por Desarrollador: Modelos personalizados podrían capturar heterogeneidad: (1) modelos específicos por categoría (GAME vs PRODUCTIVITY pueden tener dinámicas distintas), (2) modelos específicos por segmento de desarrollador (indie vs grandes estudios), y (3) transfer learning donde modelo general se fine-tunea con datos específicos del desarrollador.

8.7. Consideraciones Éticas y de Equidad

Auditoría de Sesgos: Evaluar si el modelo discrimina injustamente: (1) análisis de equidad por categoría (el modelo es igualmente preciso para FAMILY vs DATING?), (2) análisis de equidad por tipo (favorece apps gratuitas vs de pago?), (3) detección de proxies protegidos (features que correlacionan indirectamente con atributos sensibles), y (4) métricas de fairness (demographic parity, equalized odds).

Transparencia y Explicabilidad: Usuarios (desarrolladores) deben comprender las predicciones: (1) interfaces de usuario que visualizan factores de riesgo identificados, (2) recomendaciones accionables para mejorar rating predicho, (3) intervalos de confianza en predicciones (incertidumbre cuantificada), y (4) documentación clara de limitaciones del modelo.

En conclusión, este proyecto establece una base sólida para predicción de ratings en Google Play Store, pero múltiples direcciones de investigación futura podrían incrementar significativamente la capacidad predictiva, interpretabilidad y valor de negocio del sistema. La priorización de estas líneas de trabajo debería guiarse por análisis de costo-beneficio considerando impacto esperado en métricas de negocio vs esfuerzo de implementación.

Referencias

- Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). Smote: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
- Ferri, C., Flach, P. A., and Hernández-Orallo, J. (2003). Learning optimal classification trees using a binary linear program formulation. *Pattern Recognition*, 36(11):2597–2615.
- Fu, B., Lin, J., Li, L., Faloutsos, C., Hong, J., and Sadeh, N. (2013). Why people hate your app: Making sense of user feedback in a mobile app store. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1276–1284.
- Guzman, E. and Maalej, W. (2014). Ensemble methods for app review classification: An approach for software evolution. In *2014 IEEE International Conference on Automated Software Engineering (ASE)*, pages 771–776. IEEE.
- Harman, M., Jia, Y., and Zhang, Y. (2012). App store mining and analysis: Msr for app stores. In *2012 9th IEEE Working Conference on Mining Software Repositories (MSR)*, pages 108–111. IEEE.
- Hoon, L., Vasa, R., Schneider, J.-G., and Mouzakis, K. (2013). The influence of online reviews on the adoption of mobile apps: An empirical study. *Journal of Systems and Software*, 86(11):2745–2760.
- Japkowicz, N. and Stephen, S. (2002). The class imbalance problem: A systematic study. *Intelligent Data Analysis*, 6(5):429–449.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 6(4):1–21.
- Khalid, H., Shihab, E., Nagappan, M., and Hassan, A. E. (2015). What do mobile app users complain about? *IEEE Software*, 32(3):70–77.
- Martin, W., Sarro, F., Jia, Y., Zhang, Y., and Harman, M. (2016). A survey of app store analysis for software engineering. *IEEE Transactions on Software Engineering*, 43(9):817–847.

- McIlroy, S., Ali, N., Khalid, H., and Hassan, A. E. (2016). Analyzing and automatically labelling the types of user issues that are raised in mobile app reviews. In *Empirical Software Engineering*, volume 21, pages 1067–1106. Springer.
- Molnar, C. (2020). *Interpretable machine learning: A guide for making black box models explainable*. Lulu.com.
- Pagano, D. and Maalej, W. (2013). User feedback in the appstore: An empirical study. In *2013 21st IEEE International Requirements Engineering Conference (RE)*, pages 125–134. IEEE.
- Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). "why should i trust you?."explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144.
- Sokolova, M. and Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437.
- Statista (2024). Number of apps available in leading app stores as of 1st quarter 2024. <https://www.statista.com/statistics/276623/number-of-apps-available-in-leading-app-stores/>.
- Tukey, J. W. (1977). *Exploratory data analysis*, volume 2. Addison-Wesley.
- Wirth, R. and Hipp, J. (2000). Crisp-dm: Towards a standard process model for data mining. In *Proceedings of the 4th international conference on the practical applications of knowledge discovery and data mining*, volume 1, pages 29–39.
- Yin, D., Bond, S. D., and Zhang, H. (2014). Exploring the relationship between downloads and user ratings in the google play store. In *International Conference on Information Systems (ICIS)*.